

T.C OSTİM TEKNİK ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

DAĞITIK SİSTEMLER DERSİ

Hazırlayan:

Erenalp Tunay

220206041

Öğretim Görevlisi:

Prof. Dr. Ali Sebetçi

Ankara

2026

Dağıtık Sistemler Hafta-2 Deney Raporu

1. Amaç

Paralel çalışma, bir işin birden fazla işçi tarafından aynı anda yürütülerek daha kısa sürede tamamlanmasını sağlayan bir yaklaşımdır. İşçi sayısı arttıkça işin aynı oranda hızlı bitmesi beklenir; örneğin işçi sayısı iki katına çıktığında sürenin yarıya inmesi ideal durumdur. Ancak gerçek sistemlerde bu oran çoğu zaman yakalanamaz. Bu laboratuvarın amacı, işlemciyi yoğun kullanan aynı işin farklı sayıda işçiyle ölçülmesi ve hızlanmanın ideal düzeyin neden altında kaldığının ölçüm sonuçlarıyla açıklanmasıdır.

Bu amaç doğrultusunda dört durum incelenir. İlki, işin süreçlerle paralel çalıştırılmasıdır; işçi sayısı arttıkça sürenin nasıl değiştiği ve hızlanmanın hangi noktada yavaşladığı gözlenir. İkincisi, aynı işin iş parçacıklarıyla çalıştırılmasıdır; Python'un aynı anda yalnızca tek bir iş parçacığının çalışmasına izin veren yapısı nedeniyle süreçlerle arasındaki farkın nedeni açıklanır. Üçüncüsü, işin çok büyük ya da çok küçük parçalara bölünmesidir; parça boyutunun toplam süreye etkisi ölçülür. Dördüncüsü, işlemci kullanımı sınırlandırılmış bir konteyner ortamıdır; bir programın gördüğü çekirdek sayısı ile gerçekte kullanabildiği çekirdek sayısının aynı olmadığı gösterilir.

2. Sistem ve Deney Ortamı

Bu deneyde, işlemciyi yoğun kullanan aynı iş tek bir fiziksel bilgisayar üzerinde farklı işçi sayılarıyla çalıştırılarak ölçülmüştür. Bilgisayar, macOS işletim sistemli ve 16 GB belleğe sahip Apple M2 işlemcili bir sistemdir. İşlemci 4 performans ve 4 verimlilik olmak üzere 8 çekirdekten oluşur, mantıksal ve fiziksel çekirdek sayısı aynıdır. Bilgisayar deney sırasınca verimlilik yüzünden kısıtlamaların gelmemesi için prize bağlı bir şekilde gerçekleştirilmiştir.

Ölçüm programı, GIL'in açık olduğu Python 3.10.18 ile ders kökündeki sanal ortamda çalıştırılmıştır. Konteyner deneyinde Docker 29.8.0 ve python:3.12-slim imajı kullanılmıştır.

Deneyde kullanılan temel teknolojiler Python, Docker ve Git'tir. Sonuçlar CSV dosyalarına kaydedilmiş, grafik matplotlib ile çizilmiştir.

```
(.venv) erenalp@192 w2_lab1_speedup % python -m pytest -q
..........
[100%]
10 passed in 0.30s
```

Deney öncesinde pytest kontrolü yapılmış ve sonuç başarılı gözlemlenmiştir.

```
(.venv) erenalp@192 w2_lab1_speedup % python prepare_lab.py
Ad Soyad       : Erenalp Tunay
Öğrenci No     : 220206041
İşlemci hızı ölçülüyor...

=== LAB 1 YAPILANDIRMASI ===
Öğrenci           : Erenalp Tunay
Öğrenci No        : 220206041
Mantıksal işlemci : 8
Kalibrasyon       : 1 milyon dilim ≈ 0.095 s (tek çekirdek)
İş miktarı        : N_SLICES = 21.000.000 dilim
Büyük parça sayısı: GRAIN_LARGE = işçi başına 1000 görev
CPU sınırı        : CPU_LIMIT = 1.0 çekirdek

Yazıldı: lab1_config.json ve .env
```

```
(.venv) erenalp@192 Distributed-Systems-BIL_427 % python -c "import os; print(os.cpu_count())"
8
(.venv) erenalp@192 Distributed-Systems-BIL_427 % sysctl -n hw.physicalcpu hw.logicalcpu
8
8
```

3. Yöntem

Deneyde kullanılan iş, bir integralin çok sayıda küçük dilime bölünerek hesaplanmasıdır. Toplam iş miktarı, rapordaki spesifikasyonlara uygun olarak öğrenci numarasına ve bilgisayarın tek çekirdekteki hızına göre belirlenmiş, böylece tek işçiyle yaklaşık iki saniyelik bir süre hedeflenmiştir. Bu değerler yapılandırma dosyasına kaydedilmiş ve bütün ölçümlerde aynı iş miktarı kullanılmıştır.

Ölçümler dört deneyle yapılmıştır. İlk deneyde iş, süreç havuzuyla bir işçiden başlanarak işçi sayısı katlanarak artırılarak çalıştırılmış ve süre, hızlanma ve verimlilik kaydedilmiştir. İkinci deneyde aynı iş, iş parçacıklarıyla çalıştırılmış ve tek değişen havuz türü olmuştur. Üçüncü deneyde program, işlemci kullanımı sınırlandırılmış bir Docker konteynerinde çalıştırılmıştır. Dördüncü deneyde işçi başına düşen görev sayısı çok az ve çok fazla olacak şekilde değiştirilerek parça boyutunun etkisi ölçülmüştür.

Her ölçüm birkaç kez tekrarlanmış ve tekrarların ortancası alınmıştır; bu sayede tek bir ölçümdeki dalgalanmaların etkisi azaltılmıştır. Her çalıştırmada hesaplanan sonucun doğruluğu da kontrol edilmiştir. Sonuçlar CSV dosyalarına yazılmış, grafik bu dosyalardan çizilmiştir. Ölçüm sırasında bilgisayar prizde tutulmuş, yoğun programlar kapatılmıştır. Çalışmanın aşamaları Github üzerinden kayıt altına alınmıştır.

4. Deneyler ve Gözlemler

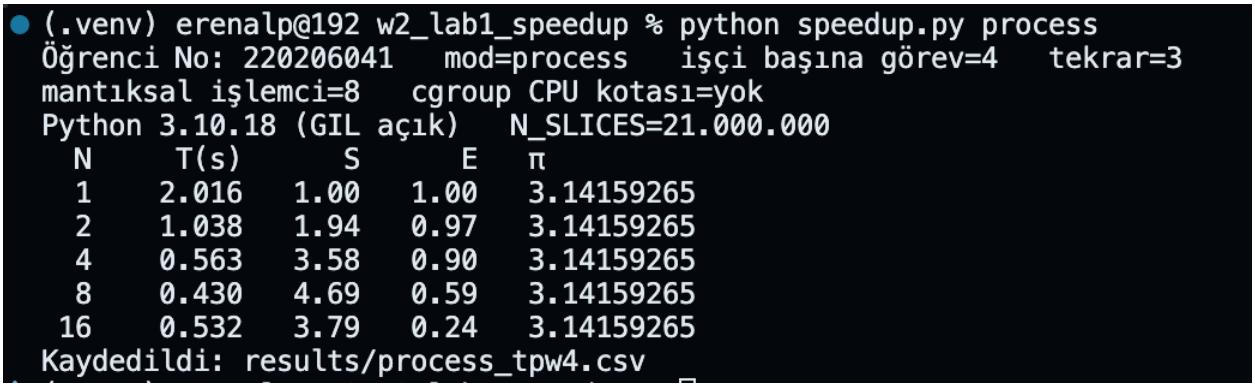
4.1 Deney 1 – Süreçlerle Hızlanma

Tahmin: İşçi sayısı arttıkça süre kısalacak, ancak hızlanma işçi sayısıyla aynı oranda artmayacaktır. Çekirdek sayısı aşıldığında artışın durması beklenmiştir.

Uygulama: Aynı iş süreç havuzuyla 1, 2, 4, 8 ve 16 işçiyle çalıştırılmıştır.

Gözlem: Süre 1 işçide yaklaşık 2,0 saniyedir ve 8 işçide 0,43 saniyeye kadar düşmüştür. 16 işçide ise süre tekrar artmıştır.

İşçi sayısı	Süre (s)	Hızlanma	Verimlilik	Karp–Flatt (e)
1	2,016	1,00	1,00	–
2	1,038	1,94	0,97	0,030
4	0,563	3,58	0,90	0,039
8	0,430	4,69	0,59	0,101
16	0,532	3,79	0,24	0,215



Açıklama: İlk iki işçide hızlanma ideale çok yakındır. Dört işçiden sonra verimlilik düşmeye başlamış, en yüksek hızlanma 8 işçide (yaklaşık 4,7 kat) elde edilmiştir. İşlemcinin 8 çekirdeği olduğu için bu beklenen bir sonuçtur; yine de 8 çekirdeğin hepsi eşit güçte olmadığından (4 performans, 4 verimlilik) 8 kata yaklaşılamamıştır. 16 işçide çekirdek sayısı aşıldığı için işçiler birbirini beklemiş ve süre yeniden artmıştır. Karp–Flatt değeri işçi sayısıyla belirgin biçimde büyümüştür (0,03'ten 0,21'e). Bu, hızlanmayı sınırlayan etkenin yalnızca işin sıralı kısmı olmadığını, işçi sayısıyla büyüyen bir ek yükün de bulunduğunu gösterir.

4.2 Deney 2 – İş Parçacıklarıyla Çalışma

Tahmin: İş parçacıklarının da süreçler gibi hızlanma sağlaması beklenmiştir.

Uygulama: Aynı iş bu kez iş parçacığı havuzuyla çalıştırılmıştır; yalnızca havuz türü değişmiştir.

Gözlem: İş parçacığı sayısı 1'den 16'ya çıkarıldığında süre yaklaşık 1,9 saniyede sabit kalmıştır. Hızlanma bütün satırlarda 1'in biraz altında (0,99) çıkmıştır. Çıktıda Python'un GIL açık olarak çalıştığı görülmektedir.

```
(.venv) erenalp@192 w2_lab1_speedup % python speedup.py thread
Öğrenci No: 220206041   mod=thread   işçi başına görev=4   tekrar=3
mantıksal işlemci=8   cgroup CPU kotası=yok
Python 3.10.18 (GIL açık)   N_SLICES=21.000.000
  N     T(s)      S      E   π
  1    1.907   1.00   1.00   3.14159265
  2    1.916   1.00   0.50   3.14159265
  4    1.919   0.99   0.25   3.14159265
  8    1.924   0.99   0.12   3.14159265
 16    1.924   0.99   0.06   3.14159265
Kaydedildi: results/thread_tpw4.csv
```

Açıklama: Python'da GIL, aynı anda yalnızca tek bir iş parçacığının hesaplama yapmasına izin verir. İşlemci yoğun işte iş parçacıkları sırayla çalıştığı için çekirdek sayısı bir fayda sağlamamıştır. Süreçlerde her işçinin kendi yorumlayıcısı ve kendi GIL'i olduğu için iş gerçekten aynı anda yürümüştür. Bunun bedeli, her sürecin ayrı başlatılması ve verilerin süreçler arasında taşınması gibi ek yüklerdir.

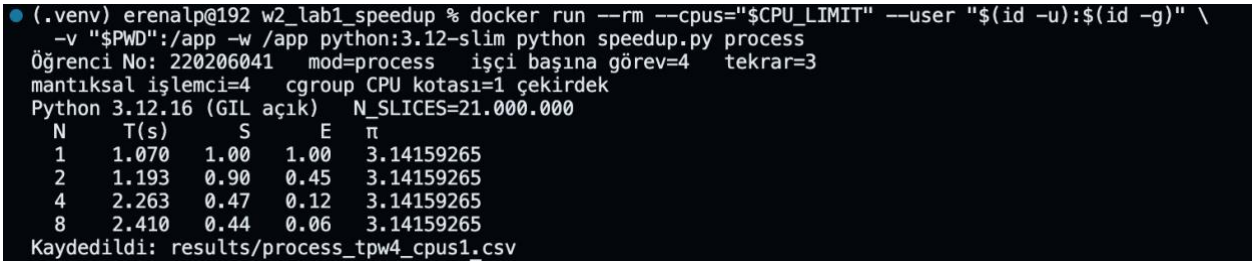
4.3 Deney 3 – Konteynerde CPU Sınırı

Tahmin: Konteynerin çekirdek sınırı 1 olduğundan, işçi sayısı artsa da hızlanma 1'in üzerine çıkmayacaktır.

Uygulama: Program, CPU kullanımı 1 çekirdekle sınırlanmış bir Docker konteynerinde çalıştırılmıştır.

Gözlem: Konteynerdeki program 4 mantıksal işlemci görmüştür, ancak CPU kotası 1 çekirdektir. Süre 1 işçide 1,07 saniye iken 2 işçide 1,19, 4 işçide 2,26 ve 8 işçide 2,41 saniyeye çıkmıştır. Hızlanma 1'in altında kalmıştır.

İşçi sayısı	Süre (s)	Hızlanma
1	1,07	1,00
2	1,19	0,90
4	2,26	0,47
8	2,41	0,44



Açıklama: Program 4 işlemci görse de kullanabildiği CPU zamanı yalnızca 1 çekirdek kadardır. Bu yüzden işçi sayısı artınca iş hızlanmamış, işçiler aynı çekirdeği paylaşmış ve süre uzamıştır. "Kaç çekirdek görüyorum" sorusu ile "kaç çekirdek kullanabilirim" sorusu bu nedenle farklıdır.

4.4 Deney 4 – Tanecik Boyutu

Tahmin: İşi çok küçük parçalara bölmek, parçaların dağıtılma maliyeti nedeniyle süreyi uzatacaktır. Çok büyük parçalarda ise dengesiz dağılımın etkisi görülebilir.

Uygulama: Aynı iş işçi başına 1 görev (çok kaba) ve 1000 görev (çok ince) ile çalıştırılmış, Deney 1'deki 4 görevle karşılaştırılmıştır.

Gözlem: Aynı işçi sayısında sürelerin karşılaştırması (saniye):

İşçi sayısı	1 görev	4 görev	1000 görev
1	2,14	2,02	2,10
2	1,04	1,04	1,10
4	0,58	0,56	0,63
8	0,47	0,43	0,97
16	0,53	0,53	2,35

```
(.venv) erenalp@192 w2_lab1_speedup % python speedup.py process --tpw 1
Öğrenci No: 220206041   mod=process   işçi başına görev=1   tekrar=3
mantıksal işlemci=8   cgroup CPU kotası=yok
Python 3.10.18 (GIL açık)   N_SLICES=21.000.000
  N     T(s)      S      E   π
  1    2.139   1.00   1.00   3.14159265
  2    1.038   2.06   1.03   3.14159265
  4    0.580   3.69   0.92   3.14159265
  8    0.468   4.57   0.57   3.14159265
 16    0.532   4.02   0.25   3.14159265
Kaydedildi: results/process_tpw1.csv
```

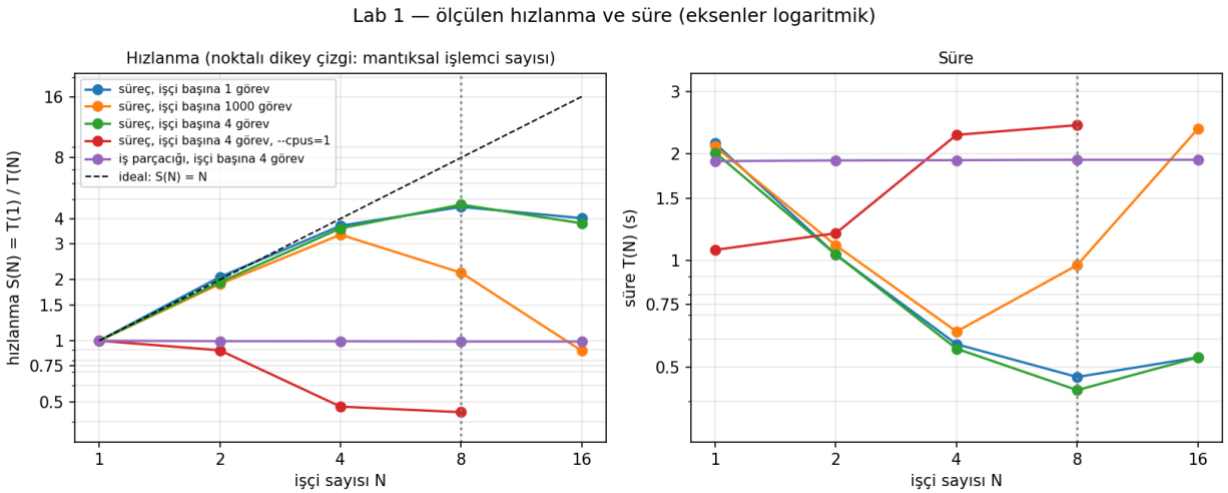
```
(.venv) erenalp@192 w2_lab1_speedup % python speedup.py process --tpw "$GRAIN_LARGE" --repeats 1
Öğrenci No: 220206041   mod=process   işçi başına görev=1000   tekrar=1
mantıksal işlemci=8   cgroup CPU kotası=yok
Python 3.10.18 (GIL açık)   N_SLICES=21.000.000
  N     T(s)      S      E   π
  1    2.098   1.00   1.00   3.14159265
  2    1.102   1.90   0.95   3.14159265
  4    0.629   3.33   0.83   3.14159265
  8    0.969   2.16   0.27   3.14159265
 16    2.353   0.89   0.06   3.14159265
Kaydedildi: results/process_tpw1000.csv
```

Açıklama: Az işçide üç boyut arasında fark yoktur. İşçi sayısı arttıkça çok ince tanecikte süre belirgin biçimde uzamıştır (16 işçide 0,53 yerine 2,35 saniye). Görev sayısı işçi sayısıyla birlikte arttığı için süreçler arası mesajlaşma ve görev dağıtımı maliyeti de artmıştır. Çok kaba

tanecikte bu çalışmada bir dezavantaj gözlenmemiştir, çünkü bütün görevlerin süresi eşittir. Görev süreleri eşit olmasaydı, uzun görev alan işçiyi diğerlerinin beklemesi beklenirdi.

4.5 Genel Grafik

Bütün ölçümler tek bir grafikte toplanmıştır. Solda hızlanma ve ideal çizgi, sağda süreler gösterilmektedir; noktalı dikey çizgi işlemcinin mantıksal çekirdek sayısını (8) işaret eder.



5. Analiz ve Tartışma

1. Deney 1'de S(N), N arttıkça nasıl değişti? Hangi N'den sonra artış durdu ya da yavaşladı? Bunu mantıksal ve fiziksel çekirdek sayılarınızla ilişkilendirin.

Hızlanma 2 işçide 1,94, 4 işçide 3,58 ve 8 işçide 4,69 olmuştur; yani 4 işçiye kadar ideale yakın artmış, 4'ten sonra yavaşlamıştır. En yüksek değer 8 işçide görülmüş, 16 işçide hızlanma 3,79'a düşmüştür. İşlemcide 8 mantıksal ve 8 fiziksel çekirdek vardır (4 performans, 4 verimlilik); bu nedenle 8'den sonra yeni çekirdek eklenmemiş, fazla işçiler yalnızca çekirdekleri paylaşmıştır. Verimlilik çekirdekleri performans çekirdeklerinden yavaş olduğu için 8 işçide bile 8 kata yaklaşılamamıştır.

2. Karp–Flatt değerleriniz (e) N arttıkça yaklaşık sabit mi kaldı, yoksa büyüdü mü? Bu, hızlanmayı sınırlayan etkenin sıralı kısım mı (Amdahl yasası), yoksa işçi sayısıyla büyüyen ek yük mü olduğu hakkında ne söylüyor?

Değer sabit kalmamış, belirgin biçimde büyümüştür: 0,03 (2 işçi), 0,04 (4), 0,10 (8), 0,21 (16). Yalnızca işin sıralı kısmı sınırlayıcı olsaydı bu değerin yaklaşık sabit kalması gerekirdi. Büyümesi, hızlanmayı işçi sayısıyla artan bir ek yükün de (süreçlerin başlatılması, görev dağıtımı, çekirdekler için rekabet) sınırladığını gösterir.

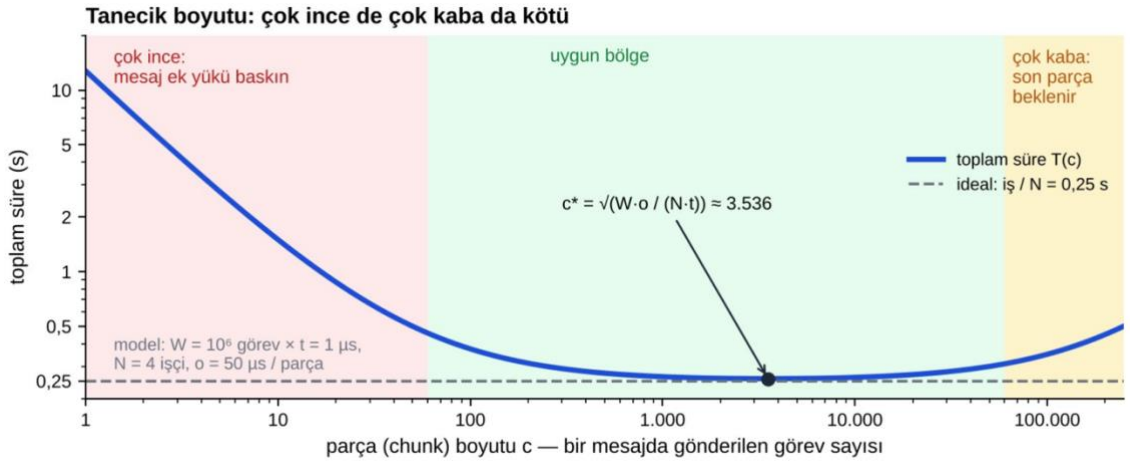
3. Deney 2'de iş parçacıklarıyla ölçülen S(N) değerlerini Deney 1 ile karşılaştırın ve farkı GIL ile açıklayın. Aynı işin süreçlerle hızlanabilmesi için hangi bedel ödendi?

İş parçacıklarında hızlanma 1, 2, 4, 8 ve 16 iş parçacığında da yaklaşık 0,99 kalmış, süre 1,9 saniye civarında sabit durmuştur. Süreçlerde ise aynı işte 8 işçiyle yaklaşık 4,7 kat hızlanma elde edilmiştir. Fark GIL'den kaynaklanır: tek süreçteki iş parçacıkları aynı anda yalnızca biri hesap yapabildiği için çekirdek sayısı fayda sağlamamıştır. Süreçlerin hızlanabilmesinin bedeli, her sürecin ayrı yorumlayıcıyla başlatılması ve süreçler arasında verinin taşınması gibi ek yüklerdir.

4. Deney 3'te konteyner içindeki program kaç mantıksal işlemci gördü ve hızlanma hangi değerde sınırlandı? İşçi sayısı CPU sınırını çok aştığında T(N) nasıl değişti? "Kaç çekirdek görüyorum?" ile "kaç çekirdek kullanabilirim?" sorularının farkını kendi ölçümünüzle açıklayın.

Konteynerdeki program 4 mantıksal işlemci görmüş, ancak CPU kotası 1 çekirdek olmuştur. Hızlanma hiçbir işçi sayısında 1'i aşmamış, 8 işçide 0,44'e düşmüştür; süre ise 1,07 saniyeden 2,41 saniyeye çıkmıştır. İşçi sayısı sınırı aştığında işçiler aynı çekirdek zamanını paylaşmış ve ek yük nedeniyle süre uzamıştır. "Kaç çekirdek görüyorum" sorusunun cevabı 4, "kaç çekirdek kullanabilirim" sorusunun cevabı ise 1'dir diyebiliriz. Program çekirdek sayısına bakarak işçi sayısı seçerse kotayı aştığı için yavaşlar.

5. Deney 4'te aynı N için üç tanecik boyutundaki T(N) değerlerini karşılaştırın. Çok ince tanecikte süreyi hangi ek yük artırdı? Çok kaba tanecikte bir dezavantaj gözlediniz mi; görevlerin süreleri eşit olmasaydı ne beklerdiniz? Ölçümlerinizi ders notlarındaki Şekil 2.7 ile ilişkilendirin.



Az işçide üç tanecik boyutu arasında anlamlı fark yoktur. 8 işçide çok ince tanecik 0,97 saniye, diğerleri 0,43–0,47 saniye; 16 işçide ise 2,35 saniyeye karşılık 0,53 saniyedir. Çok ince tanecikte işçi başına 1000 görev olduğu için toplam görev ve mesaj sayısı işçi sayısıyla artmış, süreyi artıran etken bu mesajlaşma ek yükü olmuştur. Çok kaba tanecikte (işçi başına 1 görev) dezavantaj gözlenmemiştir, çünkü bütün görevlerin süresi eşittir. Görev süreleri eşit olmasaydı, büyük bir görevi alan işçiyi diğerlerinin beklemesi beklenirdi. Bu, Şekil 2.7'deki U biçimli eğrinin ince tanecik kolunu doğrular; kaba tanecik kolu bu deneyde görünmemiştir.

6. Ölçümleriniz ne kadar tekrarlanabilir? Aynı N için üç tekrar arasındaki farkı (CSV dosyalarındaki T_runs_s sütunu) inceleyin; bu fark sonuçlarınızın yorumunu etkiliyor mu?

Üç tekrar arasındaki fark çoğu satırda %1–4 aralığındadır; en büyük farklar 1 işçili ölçümde (%6–14), 16 işçide (%9) ve konteynerde 4 işçide (%12) görülmüştür. Bu farklar süreler arasındaki büyük farkların (örneğin 2,0 s ile 0,43 s) yanında küçüktür; bu yüzden hızlanma ve ek yük yorumları değişmemiştir. Küçük farkların yorumlandığı yerlerde (örneğin 4 ve 8 işçi arasındaki hızlanma farkı) dikkatli olunmuştur. Çok ince tanecik ölçümü yalnızca bir kez yapıldığı için onun tekrarlanabilirliği değerlendirilememiştir.

6. Karşılaşılan Sorunlar, Hatalar ve Çözümler

Deney sürecinde herhangi bir problem ile karşılaşmadım.

7. Sonuç

Deney sonucunda, süreçlerle çalıştırılan işin işçi sayısı arttıkça hızlandığı, ancak bu hızlanmanın ideal düzeyde olmadığı görülmüştür. Hızlanma dört işçiye kadar ideale yakın ilerlemiş, en yüksek değere sekiz işçide ulaşmış ve çekirdek sayısı aşıldığında düşmüştür. Karp–Flatt değerinin işçi sayısıyla büyümesi, hızlanmayı yalnızca işin sıralı kısmının değil, işçi sayısıyla artan ek yükün de sınırladığını göstermiştir. İş parçacıklarıyla yapılan çalışmada ise GIL nedeniyle işçi sayısı artsa da hızlanma sağlanamamış; süreçlerin hızlanabilmesi için ise başlatma ve veri taşıma gibi ek yüklerin kabul edilmesi gerektiği anlaşılmıştır.

Konteyner deneyinde, bir programın gördüğü çekirdek sayısı ile gerçekte kullanabildiği çekirdek sayısının farklı olabileceği ve CPU sınırının üzerine çıkan işçi sayısının süreyi uzattığı gözlenmiştir. Tanecik boyutu deneyinde ise çok ince taneli işlerde mesajlaşma ek yükünün işçi sayısı arttıkça süreyi belirgin biçimde uzattığı görülmüştür. Sonuç olarak paralel hızlanmanın yalnızca işçi sayısına değil; işin yapısına, çekirdek sayısına, ek yüklere ve çalışma ortamının sınırlarına da bağlı olduğu anlaşılmıştır.

8. Kaynaklar ve Alınan Yardımlar

Ders Kaynakları ve deney talimatları.

ChatGPT - Rapordaki bilgi ve imla kontrolü, deney yapılırken ekstra araştırmaya başvurulması.

9. Ek A – Git Çalışma Geçmişi